EV Charging Sessions Data Engineering and Analytics Platform

Team: Durga Prasad Sunkara, Vamsi, Veera, Yashas

Date: 2026-02-13

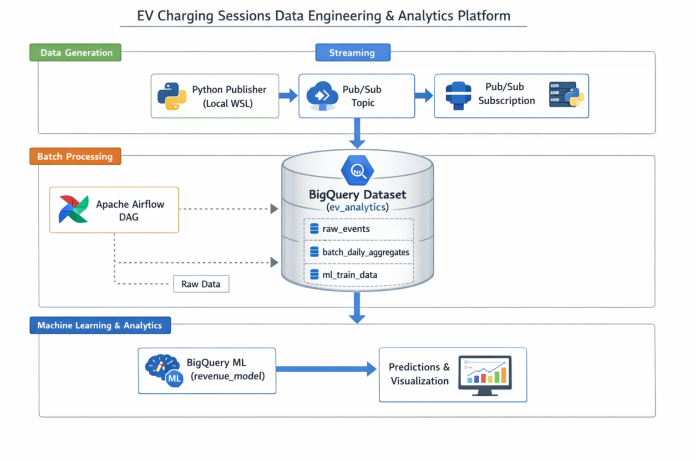
1. Project Introduction

This project implements an end-to-end data engineering and analytics pipeline for EV charging sessions. We simulate charging-session events, ingest them through a streaming layer, store and transform the data in a cloud warehouse, and train a machine learning regression model to predict total revenue. The goal is to demonstrate a complete workflow: data generation → streaming → batch aggregation → analytics → ML training → prediction and visualization.

2. Architecture Overview

The pipeline has four logical layers:

- Data Generation: Python scripts generate realistic EV session events.
- Streaming: Events flow through Kafka (local) and Google Pub/Sub (cloud).
- Warehouse: Google BigQuery stores curated tables and aggregates.
- ML & Analytics: BigQuery ML trains a regression model and SQL provides insights and visualizations.



3. Technology Stack

- Languages: Python, SQL
- Streaming: Apache Kafka (local), Google Pub/Sub (cloud)
- Orchestration / Batch: Apache Airflow (DAG-based batch aggregations)
- Warehouse & Analytics: Google BigQuery
- ML: BigQuery ML (Linear Regression)
- Infrastructure: Docker Compose, WSL (Ubuntu on Windows)
- Visualization: BigQuery Studio charts (scatter/line)

4. Dataset (EV Charging Sessions)

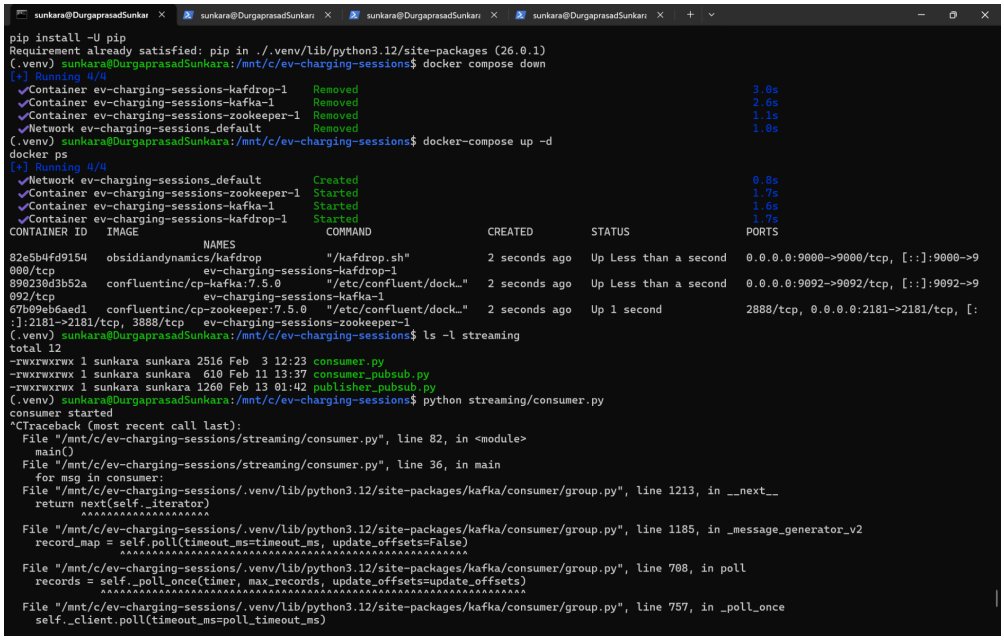
Each generated event represents a charging activity with the fields below. This schema is simple but realistic enough to support aggregations and ML features.

- event_time (timestamp)
- session_id (string)
- station_id (string)
- city (string)
- charger_type (string: fast/normal)
- energy_kwh (float)
- duration_min (float)
- price_eur (float)
- event_type (string: start/stop)

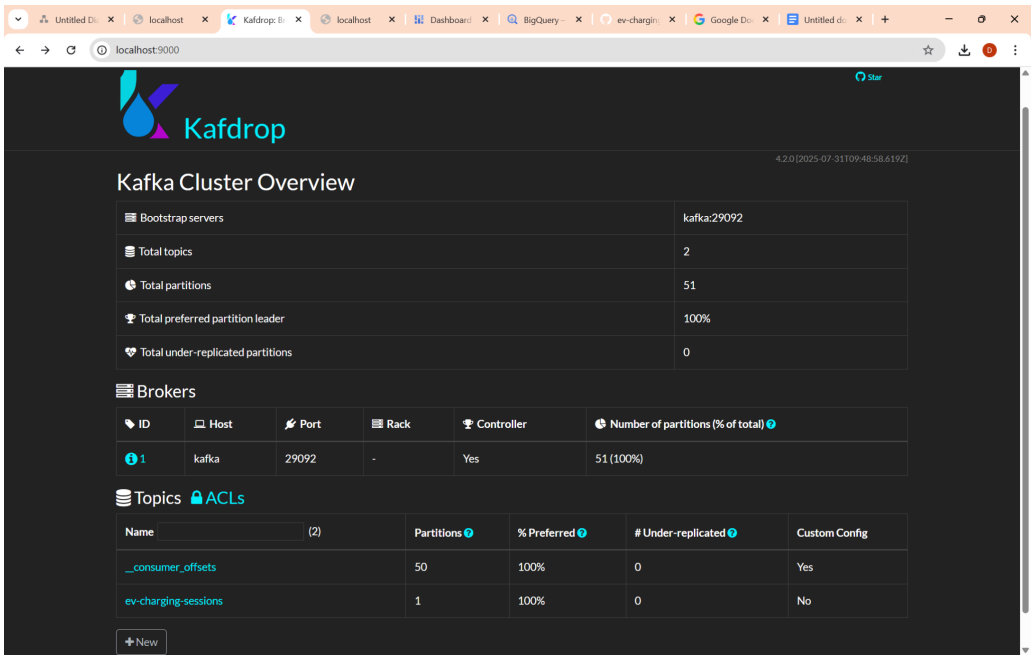
The dataset was produced continuously so that streaming consumers and batch jobs could be validated in real time.

5. Streaming Layer (Kafka + Pub/Sub)

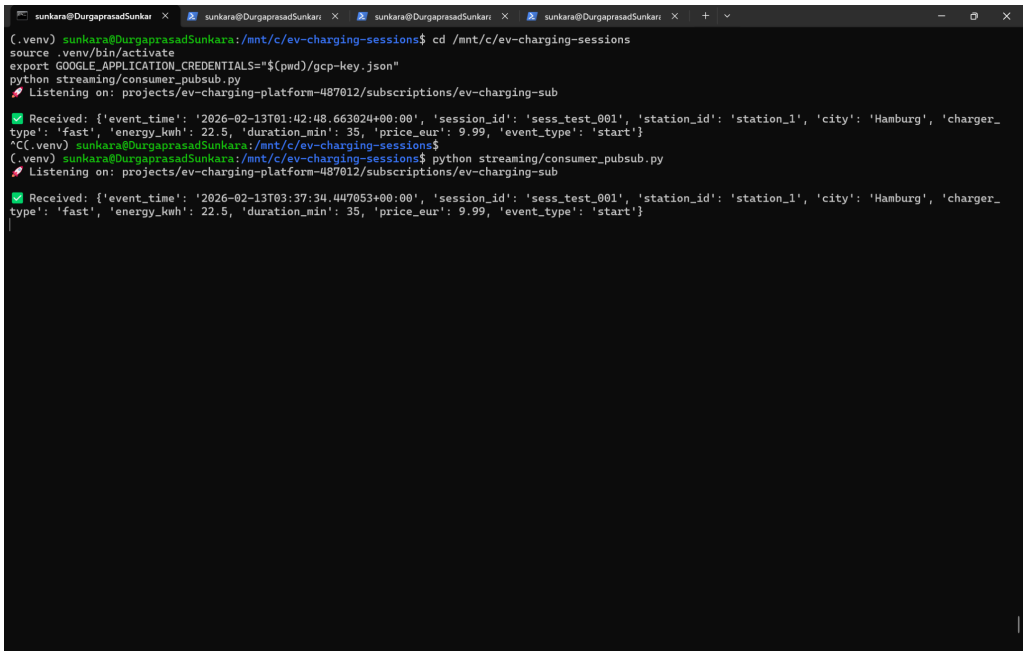
We used Kafka locally to validate a classic streaming setup and used Pub/Sub in Google Cloud to connect the data generator to cloud ingestion. Kafka was started using Docker Compose (Zookeeper + Kafka broker + Kafdrop UI). Pub/Sub publisher/consumer scripts were executed from the Python virtual environment.



Docker containers running (Kafka/Zookeeper/Kafdrop)



Kafdrop UI showing broker and topic(s) on localhost:9000



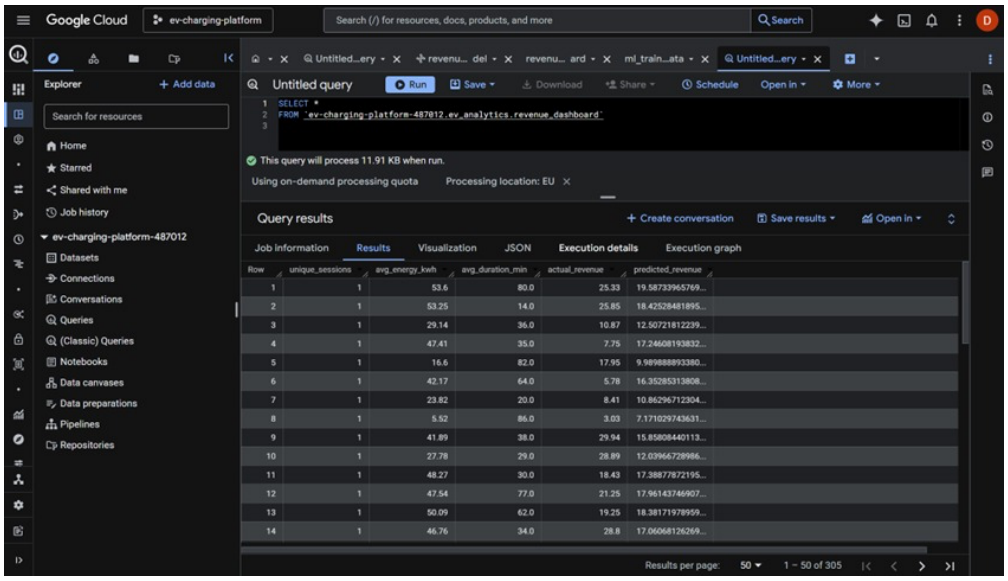
Pub/Sub consumer receiving events

6. Batch Processing (Airflow) and Aggregations

Batch processing was implemented using an Airflow DAG in the batch_airflow/ folder. The DAG is responsible for:

- Reading curated/ingested events in BigQuery (or loading raw files depending on the run).
- Producing daily aggregates used for analytics and model training.

• Creating or replacing the BigQuery table ev_analytics.batch_daily_aggregates.



Airflow UI: DAG graph

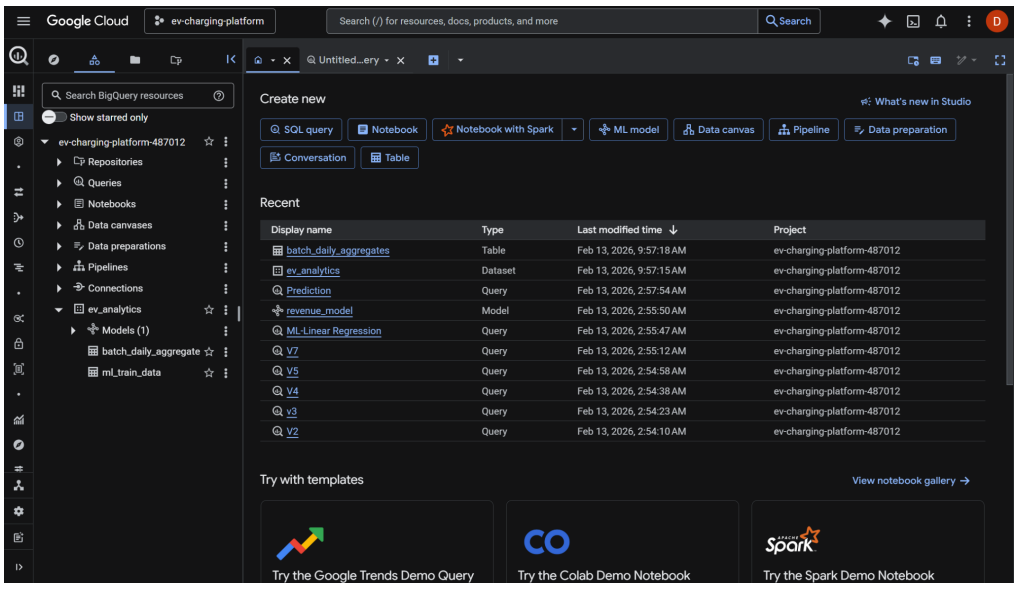
7. Cloud Warehouse (BigQuery) Tables

We used BigQuery as the central warehouse. Key objects created in dataset ev_analytics:

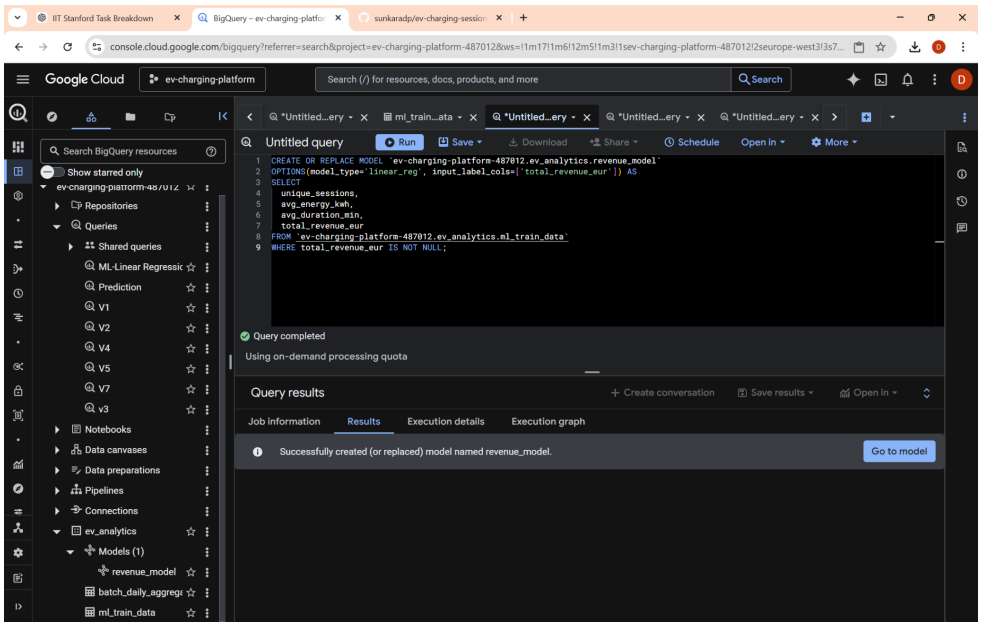
• batch_daily_aggregates: batch output table used for reporting and ML features.

• ml_train_data: final training dataset (features + label).

• revenue_model: BigQuery ML model.



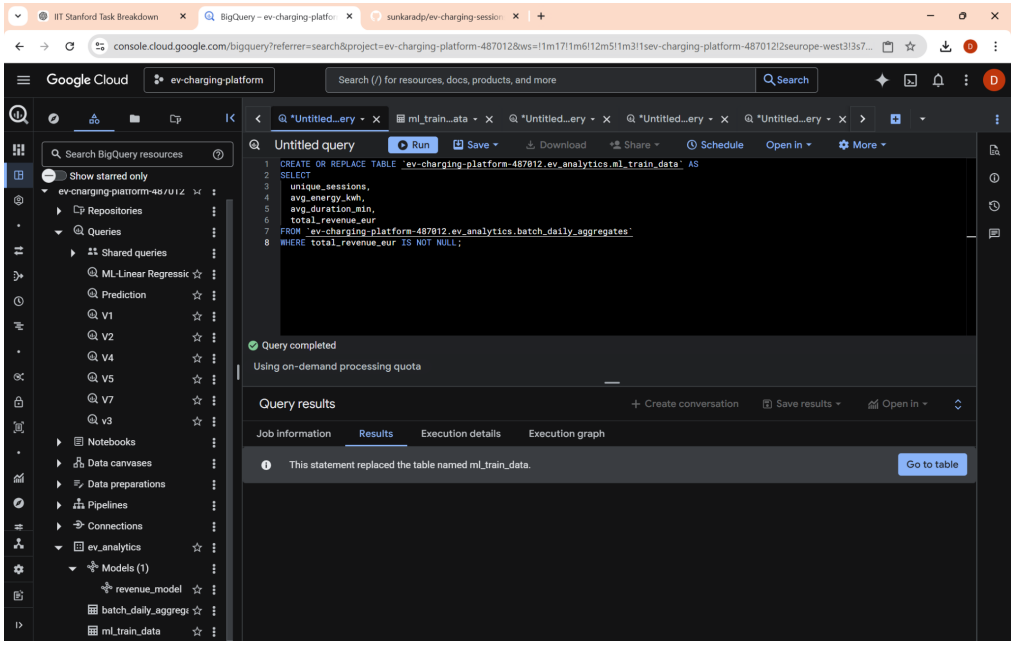
BigQuery dataset (ev_analytics) and objects



ml_train_data schema/preview (table view)

8. Feature Engineering (SQL) → ml_train_data

We created the ML-ready dataset by selecting the required columns from the batch aggregate table. This keeps the ML input stable and makes model retraining repeatable.



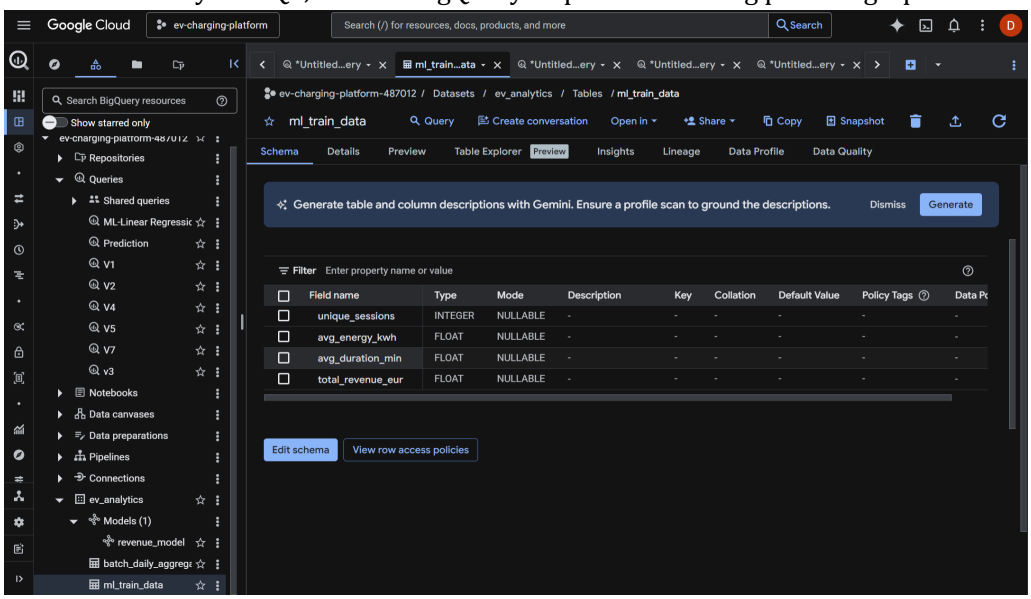
ml_train_data (SQL)

9. Machine Learning (BigQuery ML)

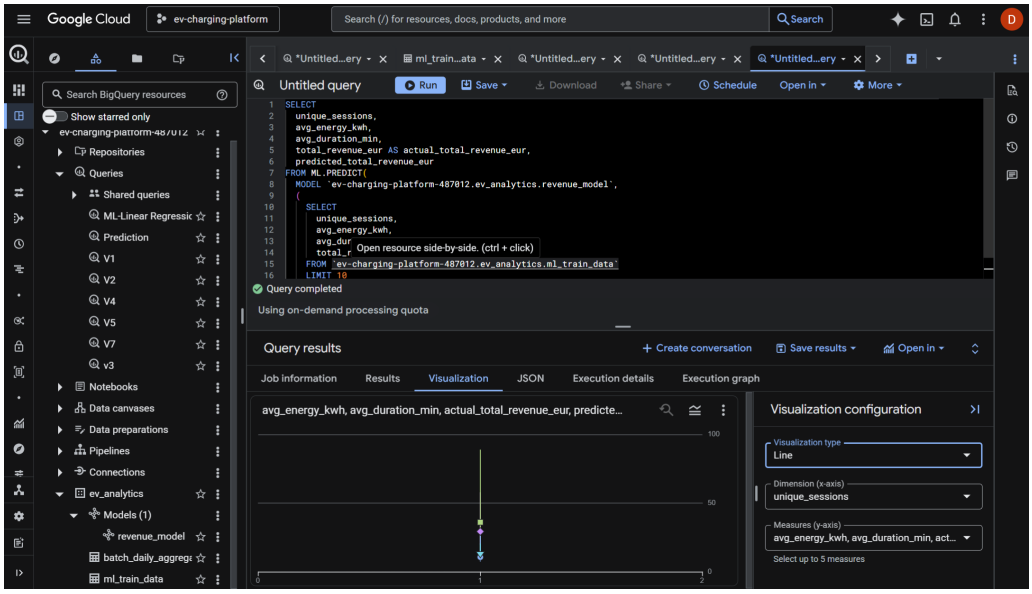
We trained a linear regression model in BigQuery ML to predict total_revenue_eur using:

- unique_sessions
- avg_energy_kwh
- avg_duration_min

The model is trained and stored as ev_analytics.revenue_model. Training and evaluation are executed directly via SQL, and the BigQuery UI provide training provide graph



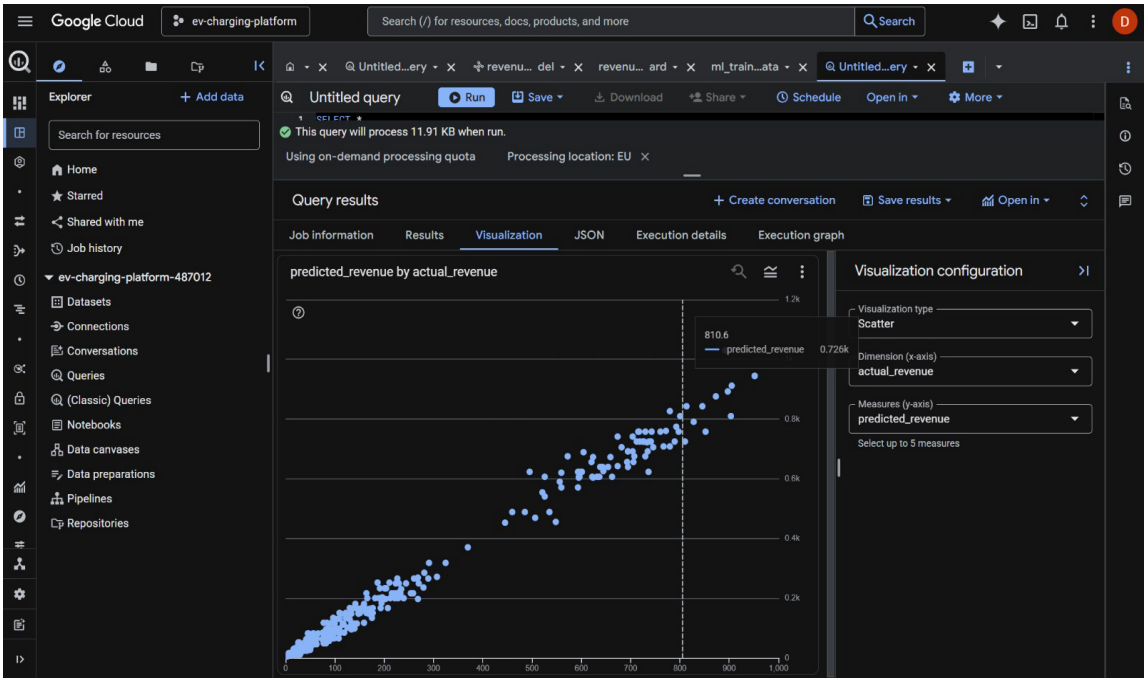
revenue_model (BigQuery ML)

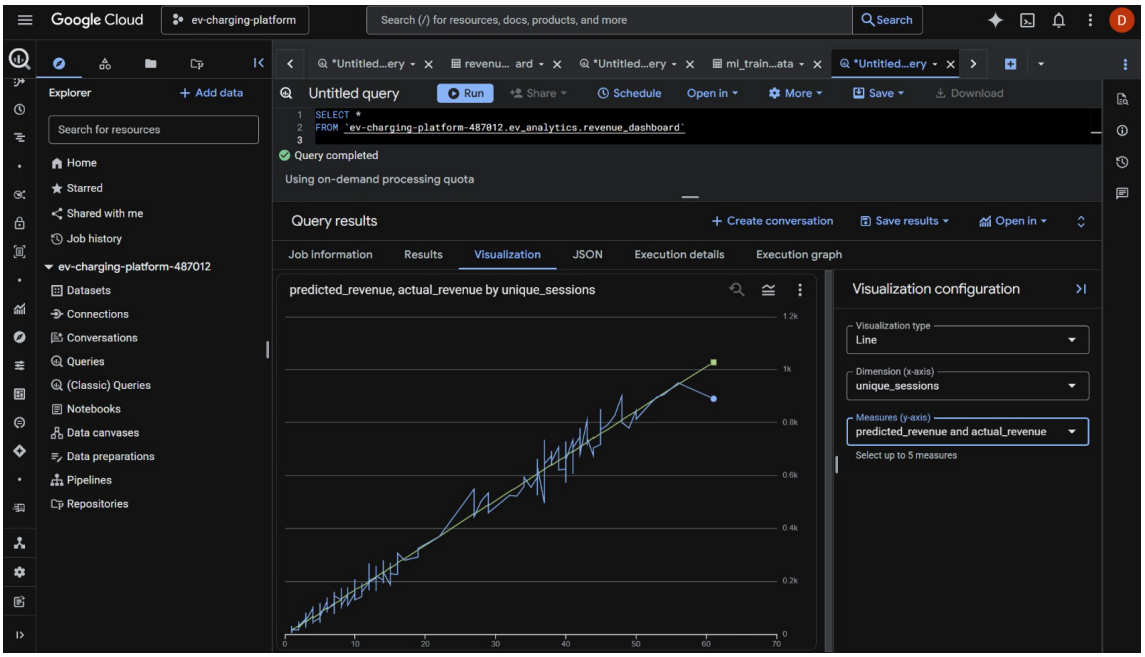


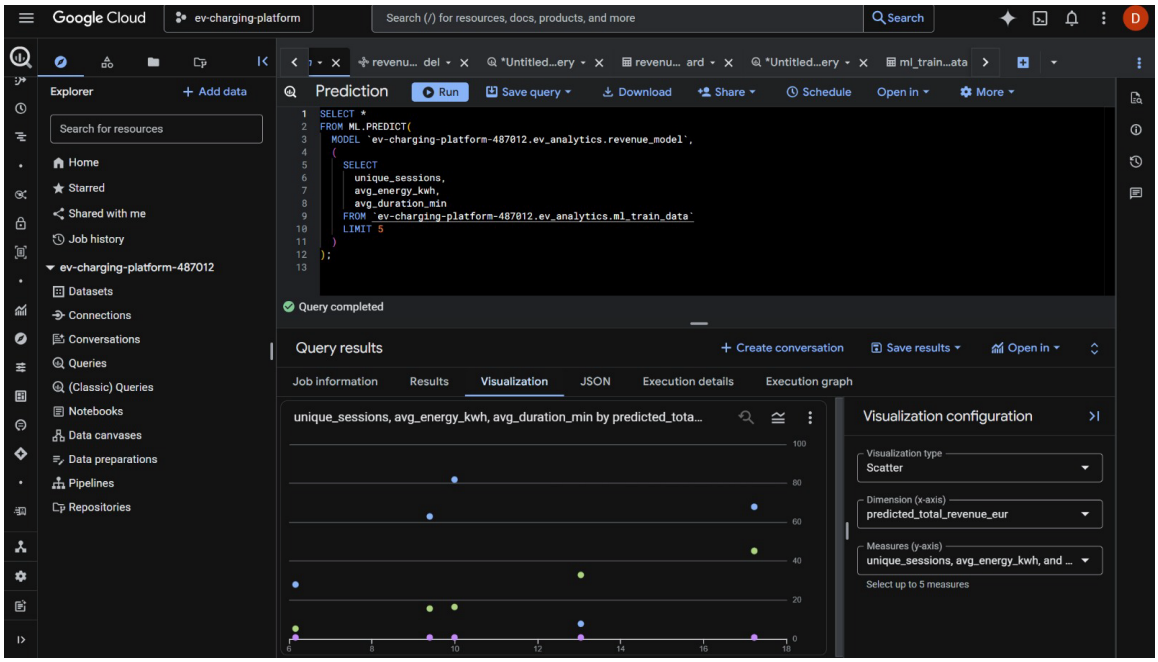
Training metrics (Loss graph)

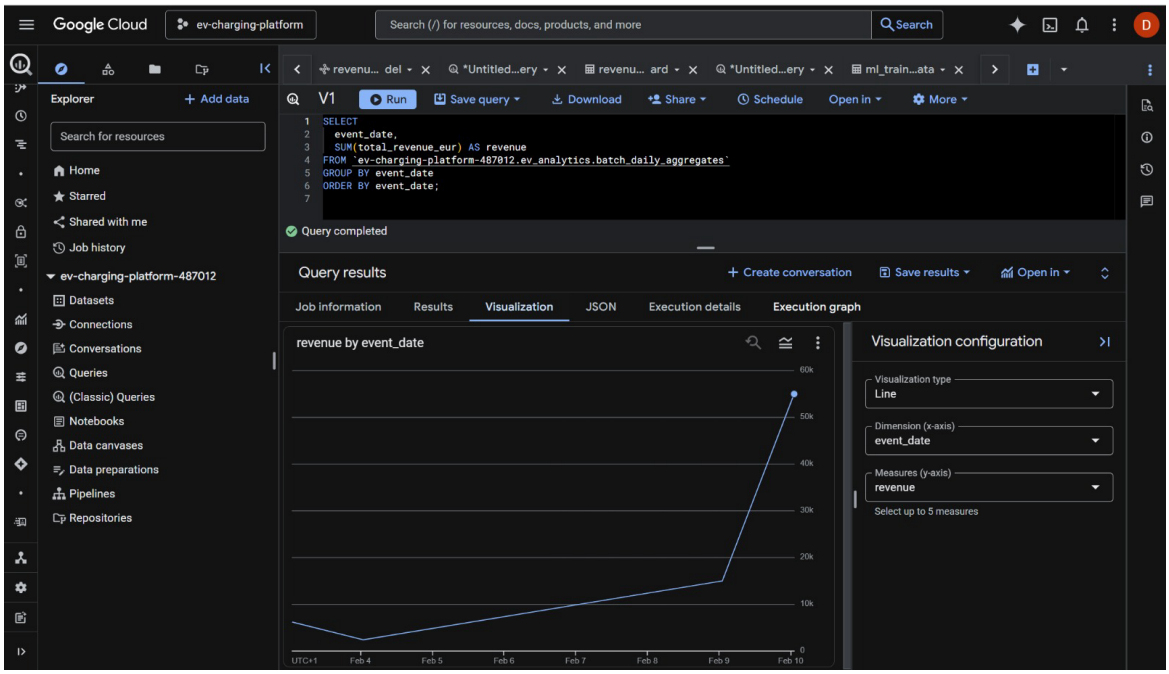
10. Predictions and Visualization (SQL)

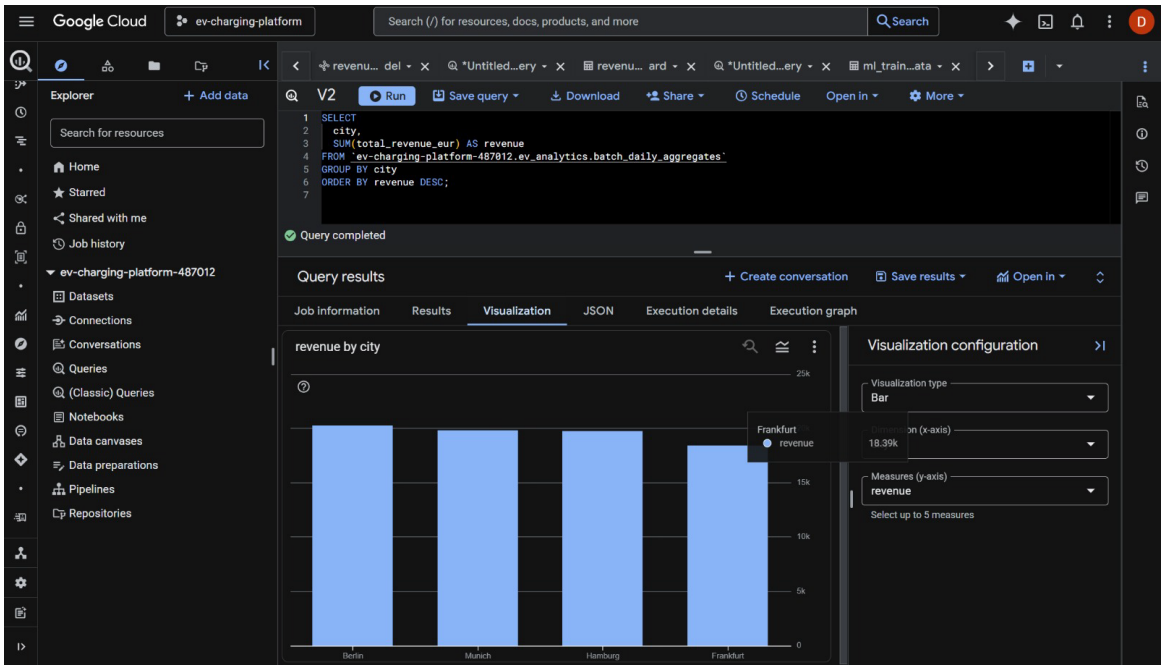
Predictions were generated using ML.PREDICT and compared with actual total revenue from the table. BigQuery Studio charts were used for quick visualization (scatter/line).





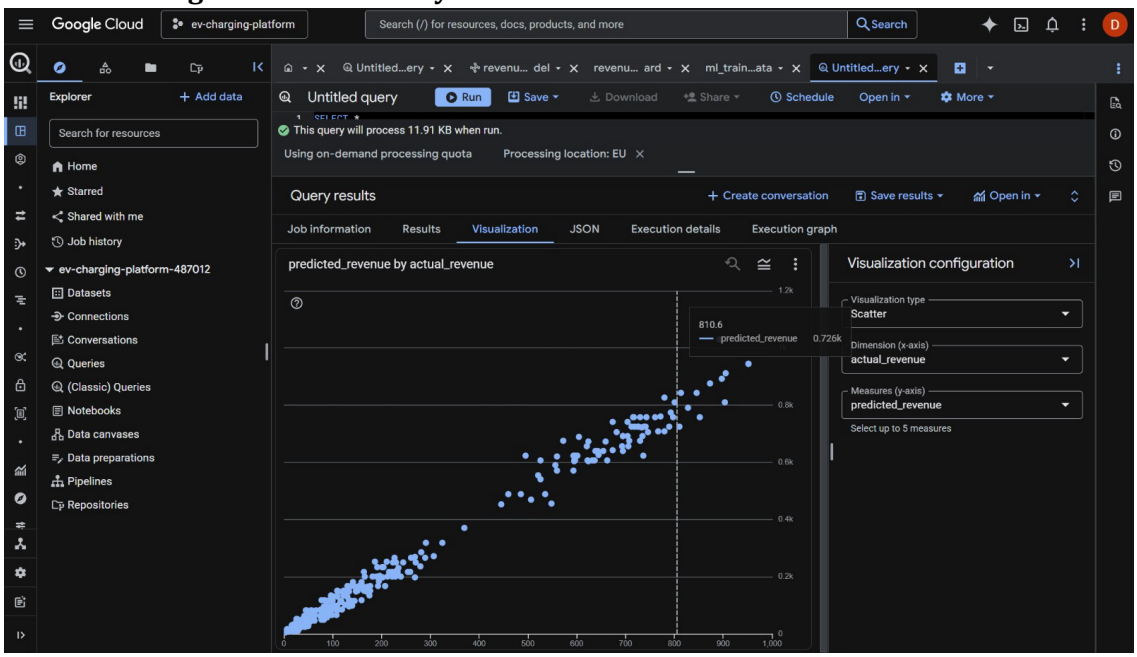

Google Cloud
ev-charging-platform
Search (/) for resources, docs, products, and more
Search
Explorer
+ Add data
Search for resources
Home
Starred
Shared with me
Job history
ev-charging-platform-487012
Datasets
Connections
Conversations
Queries
(Classic) Queries
Notebooks
Data canvases
Data preparations
Pipelines
Repositories
Prediction
Run
Save query
Download
Share
Schedule
Open in
More
SELECT *
FROM ML.PREDICT(
MODEL `ev-charging-platform-487012.ev_analytics.revenue_model`,
(
SELECT
unique_sessions,
avg_energy_kwh,
avg_duration_min
FROM `ev-charging-platform-487012.ev_analytics.ml_train_data`
LIMIT 5
)
);
Query completed
Query results
+ Create conversation
Save results
Open in
Job information
Results
Visualization
JSON
Execution details
Execution graph
unique_sessions, avg_energy_kwh, avg_duration_min by predicted_tota...
Visualization configuration
Visualization type
Scatter
Dimension (x-axis)
predicted_total_revenue_eur
Measures (y-axis)
unique_sessions, avg_energy_kwh, and ...
Select up to 5 measures


Google Cloud
ev-charging-platform
Search (/) for resources, docs, products, and more
Search
Explorer
+ Add data
Search for resources
Home
Starred
Shared with me
Job history
ev-charging-platform-487012
Datasets
Connections
Conversations
Queries
(Classic) Queries
Notebooks
Data canvases
Data preparations
Pipelines
Repositories
V1
Run
Save query
Download
Share
Schedule
Open in
More
SELECT
event_date,
SUM(total_revenue_eur) AS revenue
FROM `ev-charging-platform-487012.ev_analytics.batch_daily_aggregates`
GROUP BY event_date
ORDER BY event_date;
Query completed
Query results
+ Create conversation
Save results
Open in
Job information
Results
Visualization
JSON
Execution details
Execution graph
revenue by event_date
Visualization configuration
Visualization type
Line
Dimension (x-axis)
event_date
Measures (y-axis)
revenue
Select up to 5 measures



This screenshots are Prediction results table (rows)

11. Monitoring and Observability



Operational monitoring in our demo was primarily verified via:

- Docker container health (docker ps) for Kafka stack.
- Kafdrop UI to confirm topics/partitions.
- Pub/Sub consumer logs to confirm message flow.

• BigQuery job history and query status for batch + ML.

12. Issues Faced and Fixes

1. Wrong working directory / missing file path

Example: running python streaming/producer.py when the file did not exist. Fixed by listing files and running the correct script name and path (e.g., python streaming/publisher_pubsub.py).

2. Pub/Sub authentication (ADC) errors

Publisher/consumer failed when GOOGLE_APPLICATION_CREDENTIALS was not set or key file path was wrong. Fixed by exporting GOOGLE_APPLICATION_CREDENTIALS=$(pwd)/gcp-key.json before running Pub/Sub scripts.

3. Python env mismatch

Pub/Sub import errors occurred when scripts were run inside a different virtual environment that didn't have google-cloud-pubsub installed. Fixed by installing dependencies in the active venv (pip install google-cloud-pubsub).

4. Schema mismatch in SQL

A query referenced price_eur in batch_daily_aggregates, but the table contained total_revenue_eur. Fixed by using the correct column name.

5. Kafka consumer waiting / timeout

If the consumer starts before producer publishes, it may appear to 'hang'. Fixed by starting producer first or increasing logs and timeouts.

13. Team Contributions (4 Members)

We divided responsibilities to keep development parallel and to match the required components.

- Durga: Project integration, cloud setup (BigQuery dataset/tables), BigQuery ML model training + prediction SQL, final README/report integration.
- Vamsi: Kafka + Docker Compose setup, topic verification in Kafdrop, local streaming validation and troubleshooting.
- Veera: Batch pipeline orchestration with Airflow DAG, daily aggregation logic, and scheduling/verification steps.
- Yashas: SQL analytics and visualizations in BigQuery Studio, validation queries, and final charts/screenshots preparation.

14. Conclusion

The pipeline demonstrates an end-to-end cloud data engineering workflow: real-time event generation, streaming transport, warehouse storage, batch aggregation, analytics, and ML-based revenue prediction. The resulting artifacts (tables, model, and charts) can be recreated using the commands and SQL included in the README. The design is modular and can be extended to support additional cities, more granular station analytics, and production-grade monitoring.

15.Reference

https://github.com/sunkaradp/ev-charging-sessions.git